2.8. Herramientas y entornos para facilitar la programación, prueba y depuración/documentación del código (Criterio 2.h)

- **Resultado de Aprendizaje 2 (RA2):** *Escribe sentencias simples, aplicando la sintaxis del lenguaje y verificando su ejecución sobre navegadores Web.*

- **Criterio Curricular Oficial: CE 2.h** (*Se han utilizado herramientas y entornos para facilitar la programación, prueba y depuración/documentación del código*).

- **Ponderación del Criterio:** (**10% del RA2** | **0,5% sobre la calificación final del módulo**).

A. El Entorno de Trabajo en Desarrollo Cliente Web

Para programar en JavaScript sobre el navegador de manera profesional se requieren dos herramientas fundamentales que trabajan en sincronía:

1. **El Editor de Código / IDE:** Donde se redacta el código con resaltado de sintaxis, autocompletado y análisis estático (el estándar de facto es **Visual Studio Code**).
2. **Las DevTools del Navegador:** El conjunto de utilidades que incorpora el motor web (Chromium, Firefox, Safari) para inspeccionar la memoria, monitorizar la red y pausar la ejecución en tiempo real.

B. El Panel *Console* como Herramienta de Diagnóstico

La consola interactiva permite consultar variables y evaluar expresiones en caliente contra el contexto global de la página:

```
// 1. Clasificación por niveles de severidad (filtrables en
DevTools):
console.log("Mensaje informativo estándar.");
console.warn("Advertencia: Parámetro obsoleto en la llamada.");
console.error("Fallo crítico: No se obtuvo respuesta de la API.");

// 2. Agrupación jerárquica de trazas complejas:
console.group("Proceso de Autenticación");
console.log("Validando formato de usuario...");
console.log("Calculando hash de credenciales...");
console.groupEnd();

// 3. Inspección del árbol de propiedades de un objeto/nodo DOM:
const boton = document.querySelector("#btnEnviar");
console.log(boton); // Imprime el elemento en formato etiqueta
HTML <button>
console.dir(boton); // Imprime el elemento como objeto JavaScript
nativo con todas sus propiedades y prototipos
```

```
// 4. Rastreo de la pila de llamadas (Call Stack):
function pasoC() {
  console.trace("Ruta de llamadas recorrida hasta pasoC"); //
Muestra qué funciones llamaron a esta
}
function pasoB() { pasoC(); }
function pasoA() { pasoB(); }
pasoA();
```

Métodos Avanzados de Medición en el Objeto `console`

Para auditorías rápidas de código, el estándar de la API Console incluye métodos diagnósticos que deben dominar:

- **`console.table()`:** Renderiza arrays de objetos o matrices complejas en formato de tabla interactiva con columnas ordenables, mucho más legible que `console.log()`:

  ```
  const usuarios = [
    { id: 1, nombre: "Marta", rol: "ADMIN" },
    { id: 2, nombre: "Javier", rol: "USER" }
  ];
  console.table(usuarios);
  ```

- **`console.assert(condición, mensaje)`:** Permite realizar pruebas unitarias rápidas integradas en el script. Si la condición se evalúa como `false`, imprime el mensaje en rojo con traza de error sin detener la ejecución de la página:

  ```
  const edad = -5;
  console.assert(edad >= 0, "Error crítico: La edad no puede
  ser negativa");
  ```

- **`console.count(etiqueta)` y `console.countReset(etiqueta)`:** Lleva la cuenta automática de cuántas veces se ha alcanzado una línea de código o se ha invocado una función concreta, ideal para detectar renderizados o iteraciones innecesarias.

  ### Ejemplo Básico: Detección de Invocaciones y Reinicio de Turno

  En este caso se audita cuántas veces se ejecuta una función:

  ```
  function procesarIntento(usuario) {
    // Incrementa y muestra el contador para la etiqueta
  indicada
    console.count(`Intentos de login [${usuario}]`);
  ```

```
}

// Simulamos accesos del primer usuario
procesarIntento("marcos"); // Salida: Intentos de login
[marcos]: 1
procesarIntento("marcos"); // Salida: Intentos de login
[marcos]: 2
procesarIntento("marcos"); // Salida: Intentos de login
[marcos]: 3

// El usuario supera el límite y reiniciamos el contador de
ese identificador exacto
console.countReset("Intentos de login [marcos]");

procesarIntento("marcos"); // Salida: Intentos de login
[marcos]: 1 (comienza de nuevo)
```

Ejemplo Algorítmico: Auditoría de Eficiencia en Bucles

Comparar cuántas iteraciones ejecuta un algoritmo con salida prematura (break) frente a uno sin optimizar:

```
const usuarios = ["Ana", "Carlos", "Marta", "Luis", "Elena"];
const BUSCADO = "Carlos";

console.log("--- Búsqueda sin optimizar ---");
for (let i = 0; i < usuarios.length; i++) {
  console.count("Iteración bucle completo");
  if (usuarios[i] === BUSCADO) {
    // Encuentra el dato pero sigue iterando innecesariamente
  }
}
// Salida: Iteración bucle completo: 1, 2, 3, 4, 5 (5 ciclos)

console.countReset("Iteración bucle optimizado");

console.log("--- Búsqueda optimizada con break ---");
for (let i = 0; i < usuarios.length; i++) {
  console.count("Iteración bucle optimizado");
  if (usuarios[i] === BUSCADO) {
    break; // Corte inmediato tras encontrar el dato
  }
}
// Salida: Iteración bucle optimizado: 1, 2 (se detiene en el
segundo ciclo)
```

Puntos clave a recordar:

1. **Sensible a mayúsculas y espacios:** console.count("test") y console.count("Test") son dos contadores completamente independientes.
2. **Error al reiniciar etiquetas inexistentes:** Si se invoca console.countReset("etiqueta") sobre una clave que nunca ha sido llamada con console.count(), el navegador emite una advertencia (*warning*) en la consola: Count for 'etiqueta' does not exist.

C. Depuración Profesional en el Panel *Sources*

Confiar únicamente en console.log() para solucionar fallos es ineficiente en aplicaciones complejas. La técnica profesional se basa en **pausar el hilo de JavaScript** mediante puntos de interrupción y examinar el estado de la memoria paso a paso.

1. Tipos de Puntos de Interrupción (*Breakpoints*)

- **Line Breakpoint (Punto de línea):** Se añade haciendo clic sobre el número de línea en el panel *Sources*. El programa se detiene en seco justo antes de ejecutar esa línea.
- **Conditional Breakpoint (Punto condicional):** Se hace clic derecho sobre el número de línea → *Add conditional breakpoint...* Solo pausa la ejecución si se cumple una expresión booleana (por ejemplo: i === 500 o usuario.rol === 'ADMIN'). Es imprescindible para depurar bucles masivos sin tener que pausar manualmente en cada vuelta.
- **Logpoint (Registro sin alterar código):** Hace clic derecho → *Add logpoint...* Imprime un mensaje o variable en la consola al pasar por esa línea sin modificar físicamente el archivo .js y sin detener el flujo.
- **Instrucción debugger; en el código fuente:** Palabra reservada nativa de JavaScript. Si las DevTools están abiertas, el navegador se pausa automáticamente en esa línea exacta:

```
function calcularDescuento(cliente, base) {
  if (cliente.antiguedad > 5) {
    debugger; // Pausa forzosa del navegador si DevTools está abierto
    return base * 0.80;
  }
  return base;
}
```

2. Puntos de Interrupción Especializados en DevTools (Más allá de los breakpoints de línea)

En el panel *Sources* de las DevTools, además de los breakpoints sobre líneas de código, existen tres tipos de interrupciones automáticas esenciales para el cliente web:

- **XHR/fetch Breakpoints:** Permiten pausar el motor en el momento exacto en que se dispara una petición asíncrona hacia una URL que contenga un patrón concreto (por ejemplo, pausar cuando se llame a /api/pedidos).
- **DOM Mutation Breakpoints:** Se activan desde el panel *Elements* haciendo clic derecho sobre un nodo HTML → *Break on*:
    - *Subtree modifications:* Pausa si un script añade o elimina nodos hijos.
    - *Attribute modifications:* Pausa si un script cambia una clase CSS, un estilo en línea o un atributo.
    - *Node removal:* Pausa en la línea exacta de JavaScript que elimina ese elemento del documento.
- **Event Listener Breakpoints:** Ubicados en el acordeón lateral de *Sources*. Permiten marcar categorías enteras de eventos (por ejemplo, marcar Mouse > click o Keyboard > keydown) para que el navegador se detenga automáticamente en la primera línea de cualquier manejador cuando el usuario interactúe con la página.

3. Navegación Paso a Paso del Hilo de Ejecución

Cuando el programa se detiene en un breakpoint, se controla el avance con los botones superiores o el teclado:

- **Resume (F8):** Reanuda la ejecución continua hasta encontrar el siguiente punto de interrupción o finalizar el script.
- **Step Over (F10):** Ejecuta la línea actual y pasa a la siguiente; si la línea contiene una llamada a una función, la ejecuta por completo sin entrar en su interior.
- **Step Into (F11):** Si la línea actual contiene una función, "entra" dentro de ella para depurarla línea a línea.
- **Step Out (Shift + F11):** Si nos encontramos dentro de una función, ejecuta el resto de sus instrucciones de golpe y se detiene en la línea exterior que la invocó.

4. Pila de Llamadas (*Call Stack*) y Expresiones Vigiladas (*Watch*)

- **Call Stack:** Muestra la jerarquía de funciones que se han ido apilando para llegar a la línea actual. Si una función falla, el *Call Stack* permite hacer clic hacia atrás en la lista para ver exactamente qué archivo y función la llamaron y con qué argumentos.
- **Watch Expressions:** Permite escribir variables o expresiones complejas para que el panel muestre su valor en vivo en cada paso de la depuración.

D. La distinción entre errores de sintaxis y errores de lógica en la consola

Tipos de mensajes reporta la consola de depuración del navegador:

1. **Errores sintácticos (*SyntaxError*):** Detectados por el intérprete al cargar el script (paréntesis sin cerrar, comillas incompletas, palabras reservadas mal escritas). El script no llega a ejecutarse o se detiene en la línea del fallo.
2. **Errores en tiempo de ejecución (*Runtime Errors* / *TypeError*, *ReferenceError*):** Ocurren cuando el código intenta acceder a variables inexistentes o invocar métodos sobre tipos incompatibles (por ejemplo, llamar a una función que no ha sido definida).

3. **Errores de lógica:** El script se ejecuta sin arrojar mensajes rojos en la consola, pero produce resultados numéricos o visuales incorrectos (aquí es donde hacemos uso de puntos de interrupción y seguimiento de variables paso a paso).

ACTIVIDAD PRÁCTICA: DEPURACIÓN EN EL NAVEGADOR

Criterio Evaluado: CE 2.h (Calificación sobre 10)

Formato de Entrega: Grabación de vídeo individual (Máximo 5 minutos, continuo y con audio explicativo).

1. Código Fuente del Laboratorio

`index.html` (Estructura base)

html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Laboratorio CE 2.h: Herramientas y Depuración Integral</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <div class="panel">
    <h2>Laboratorio Integral de Depuración de Código</h2>
    <p>Abre las DevTools (F12) en las pestañas <strong>Sources</strong>
y <strong>Elements</strong> antes de interactuar.</p>

    <div class="fila">
      <label for="campoUnidades">Unidades a procesar (ej: 5):</label>
      <input type="text" id="campoUnidades" value="5">
    </div>

    <div class="fila">
      <label for="campoPrecio">Precio unitario base (€):</label>
      <input type="text" id="campoPrecio" value="20">
    </div>

    <div class="botones-accion">
      <button id="btnEjecutar">1. Ejecutar y Depurar Algoritmo</button>
      <button id="btnModificarDOM">2. Provocar Mutación DOM</button>
    </div>
```

```
    <div id="pantallaSalida" class="visor">Esperando orden de
ejecución...</div>

    <div id="zonaMonitorizada" class="contenedor-dinamico">
      <span id="nodoObjetivo">Nodo base pendiente de
modificación</span>
    </div>
  </div>
  <script src="depurador_lab.js"></script>
</body>
</html>
```

Usa el código con precaución.

`estilos.css` (Hoja de estilos externa)

CSS

```
body {
  font-family: monospace;
  margin: 30px;
  background-color: #0f172a;
  color: #f8fafc;
}
.panel {
  max-width: 650px;
  background: #1e293b;
  padding: 25px;
  border-radius: 8px;
  border: 1px solid #334155;
}
h2 { color: #38bdf8; margin-top: 0; }
.fila { margin-bottom: 12px; }
label { display: block; margin-bottom: 4px; color: #94a3b8; }
input {
  width: 95%; padding: 8px; background: #0f172a;
  border: 1px solid #475569; color: #f8fafc; border-radius: 4px;
}
.botones-accion { display: flex; gap: 10px; margin-top: 15px; }
button {
  padding: 10px 16px; background-color: #0284c7; color: white;
  border: none; font-weight: bold; cursor: pointer; border-radius: 4px;
}
button:hover { background-color: #0369a1; }
#btnModificarDOM { background-color: #7c3aed; }
#btnModificarDOM:hover { background-color: #6d28d9; }
.visor {
  margin-top: 20px; padding: 15px; background: #020617;
  border-radius: 4px; border-left: 4px solid #38bdf8;
```

```
  line-height: 1.6; white-space: pre-wrap;
}
.contenedor-dinamico {
  margin-top: 15px; padding: 10px; background: #1e293b;
  border: 1px dashed #64748b; min-height: 30px;
}
```

Usa el código con precaución.

depurador_lab.js (Código defectuoso entregado)

javascript

```
function calcularRecargo(base) {
  const FACTOR_AJUSTE = 1.05;
  return base * FACTOR_AJUSTE;
}

function procesarLote() {
  const unidades = document.getElementById("campoUnidades").value;
  const precioBase =
Number(document.getElementById("campoPrecio").value);
  const visor = document.getElementById("pantallaSalida");

  acumuladoTotal = 0;

  debugger;

  const precioConRecargo = calcularRecargo(precioBase);

  const lotes = [10, 20, 30, 40, 50];
  let sumaLotes = 0;

  for (let i = 0; i <= lotes.length; i++) {
    sumaLotes += lotes[i];
  }

  let contadorAuditoria = 0;
  for (let k = 0; k < 10000; k++) {
    contadorAuditoria += 1;
  }

  acumuladoTotal = (precioConRecargo * unidades) + sumaLotes;

  visor.innerHTML = `
    Subtotal lotes: ${sumaLotes}<br>
    Precio con recargo (unitario): ${precioConRecargo.toFixed(2)} €<br>
    Total final computado: ${acumuladoTotal} €<br>
    Auditoría masiva completada: ${contadorAuditoria} ciclos<br>
```

```
    ¿Se contaminó window?: ${typeof window.acumuladoTotal !==
"undefined" ? "SÍ (Fuga)" : "NO"}
  `;
}

function alterarEstructuraDOM() {
  const contenedor = document.getElementById("zonaMonitorizada");
  const nodo = document.getElementById("nodoObjetivo");

  contenedor.setAttribute("data-estado", "actualizado");

  const nuevoElemento = document.createElement("p");
  nuevoElemento.textContent = `Elemento inyectado a las ${new
Date().toLocaleTimeString()}`;
  nuevoElemento.style.color = "#38bdf8";
  contenedor.appendChild(nuevoElemento);

  if (nodo) {
    nodo.remove();
  }
}

document.getElementById("btnEjecutar").addEventListener("click",
procesarLote);
document.getElementById("btnModificarDOM").addEventListener("click",
alterarEstructuraDOM);
```

2. Guion Práctico del Vídeo (Pasos que debes grabar)

- **Fase 1: Ámbitos y Control Paso a Paso (F10 / F11)**
    1. Abre las DevTools (`F12`), ve a la pestaña **Sources** y haz clic en el botón *"1. Ejecutar y Depurar Algoritmo"*. El código se parará en la instrucción `debugger`.
    2. Muestra el panel derecho **Scope** y explica que `unidades` es un *String* ("5") y que `acumuladoTotal` no está declarada, fugándose a `window`.
    3. Muestra cómo lo corriges en directo aplicando `Number(...)` y añadiendo `"use strict";` junto a la declaración `let` al inicio del archivo.
    4. Avanza con **F10 (Step Over)** hasta llegar a `calcularRecargo`. Pulsa **F11 (Step Into)** para entrar a la función, inspecciona `FACTOR_AJUSTE` y sal pulsando **Shift + F11 (Step Out)**.
- **Fase 2: Watch y Punto Condicional (Conditional Breakpoint)**
    1. Añade la expresión `lotes[i]` a la pestaña lateral **Watch**. Avanza con **F10** en el bucle hasta comprobar que en la vuelta `i === 5` devuelve `undefined`, rompiendo la suma a `NaN`. Muestra cómo lo corriges cambiando `<=` por `<`.

2. Haz clic derecho sobre el número de línea de `contadorAuditoria += 1;` dentro del bucle masivo, selecciona **Add conditional breakpoint...** y escribe la condición: `k === 7420`.
3. Pulsa **F8 (Resume)** y demuestra que el navegador no se bloquea, sino que se pausa exactamente cuando `k` vale 7420.

- **Fase 3: Depuración por Mutaciones del DOM**
  1. Dirígete al panel **Elements** de las DevTools, localiza el contenedor `<div id="zonaMonitorizada">`, haz clic derecho sobre la etiqueta y selecciona **Break on**. Activa las tres opciones: *Subtree modifications*, *Attribute modifications* y *Node removal*.
  2. Regresa a la página y haz clic en *"2. Provocar Mutación DOM"*.
  3. Muestra cómo el navegador congela de inmediato el hilo en la línea de JavaScript exacta que modifica el atributo o añade el nuevo nodo.
- **Fase 4: Captura mediante Event Listener Breakpoints**
  1. Desactiva los breakpoints del paso anterior. En la columna derecha de **Sources**, despliega la sección **Event Listener Breakpoints**.
  2. Abre la categoría **Mouse** y marca la casilla **click**.
  3. Haz clic en cualquiera de los botones de tu página web y demuestra cómo el depurador caza la interacción frenando la primera línea de ejecución del manejador sin necesidad de saber previamente dónde estaba programado.

3. Rúbrica de Autoevaluación (CE 2.h)

Completa la práctica siguiendo el guion, marca las casillas de verificación de los logros demostrados en el vídeo, calcula los subtotales de cada bloque y obtén tu calificación final en la sección de cierre.

Fase / Indicador de Logro (Demostrado con audio y vídeo)	
BLOQUE 1: FASE 1 - CONTROL DE FLUJO Y ÁMBITOS	**Puntuación**
▢ **Muestra la parada en `debugger`:** Inicia la depuración deteniendo el programa y explicando el panel *Scope*.	2,5 pts
▢ **Corrige el tipo String:** Identifica el error de conversión del texto y aplica correctamente el envoltorio `Number()`.	2,5 pts

▢ **Soluciona la fuga de ámbito:** Activa el modo estricto (`"use strict";`) y define la variable local con la palabra clave `let`.	2,5 pts
▢ **Domina el flujo manual:** Ejecuta con soltura los comandos de salto y avance por líneas (**F10, F11 y Shift+F11**).	2,5 pts
SUBTOTAL FASE 1:	**_____ / 10**

BLOQUE 2: FASE 2 - WATCH Y BREAKPOINT CONDICIONAL	**Puntuación**
▢ **Diagnostica el Array con Watch:** Realiza el seguimiento de `lotes[i]` en el panel visualizando el desbordamiento `undefined`.	2,5 pts
▢ **Corrige el bucle:** Modifica el operador relacional crítico de `<=` a `<` solucionando el cálculo fallido por el valor `NaN`.	2,5 pts
▢ **Configura el Breakpoint Condicional:** Implementa la expresión lógica exacta `k === 7420` directamente sobre la línea del bucle.	2,5 pts
▢ **Demuestra la parada exacta:** Reanuda con **F8** y enseña que la ejecución se detiene únicamente en el ciclo especificado.	2,5 pts
SUBTOTAL FASE 2:	**_____ / 10**

BLOQUE 3: FASES 3 Y 4 - BREAKPOINTS AVANZADOS	**Puntuación**
▢ **Configura las mutaciones del DOM:** Activa las tres directivas de interrupción (*Subtree*, *Attribute* y *Removal*) en el nodo objetivo.	2,5 pts
▢ **Intercepta cambios asíncronos:** Captura el momento exacto en el que el script altera la estructura visual de la interfaz.	2,5 pts
▢ **Activa Event Listener Breakpoints:** Habilita el detector automático de escuchadores para eventos de tipo **click** en el ratón.	2,5 pts

▢ **Explica la captura del evento:** Intercepta la acción del usuario en el navegador mostrando el inicio limpio de la función.	2,5 pts
SUBTOTAL FASE 3:	**_____ / 10**
Calificación Final CE 2.h **Media de las Fases: (Subtotal 1 + Subtotal 2 + Subtotal 3) / 3**	**_____ / 10**